Experiment 8: Iterative Deepening Search

CSA1717 – Artificial Intelligence

Name: Hemalakshmi H

Reg.No.: 192571049

Aim:

To implement **Iterative Deepening Search (IDS)**, a search strategy that repeatedly performs Depth-Limited Search (DLS) with increasing depth limits until the goal node is found.

Algorithm:

Iterative Deepening Search (IDS)

1. Start with depth limit = 0
2. Perform Depth-Limited Search (DLS) with the current depth limit
3. If goal is found → STOP
4. Else increase depth limit by 1
5. Repeat until goal is found

Depth-Limited Search (DLS)

1. If depth limit = 0 → return failure
2. If current node = goal → return success
3. For each child of current node:
    - Recursively call DLS with depth limit − 1
4. If any recursive call succeeds → return success
5. Else return failure

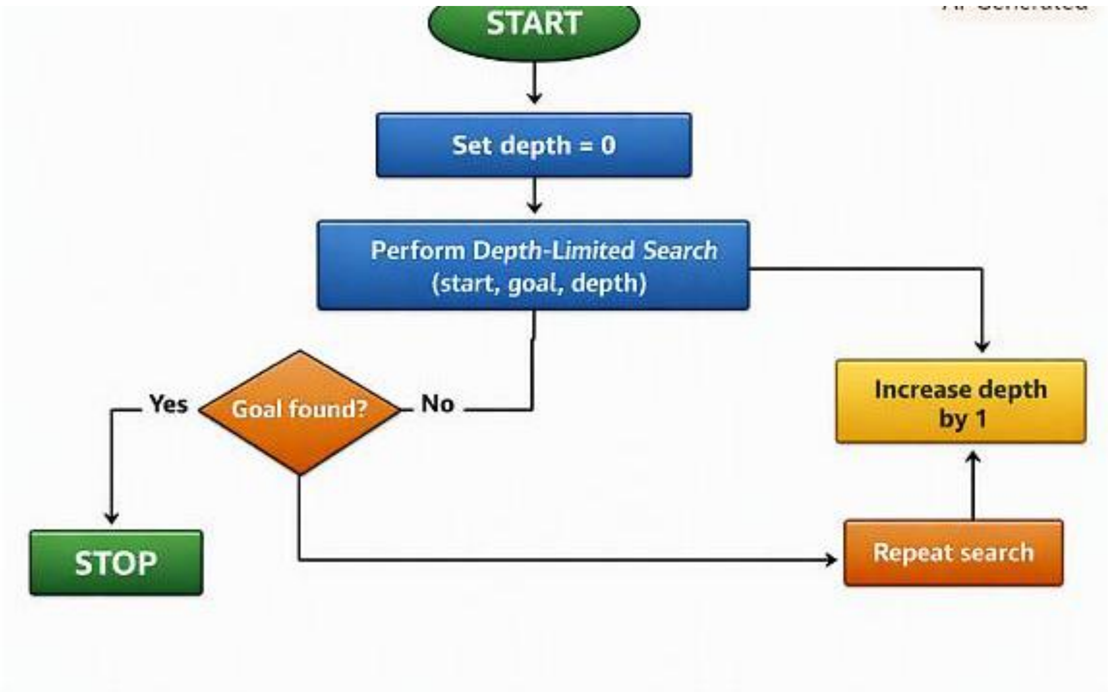
Source Code:

```
def dfs_limited(node, goal, depth_limit, graph):
    if depth_limit == 0:
        return None
```

```
    if node == goal:
        return [node]

    for child in graph.get(node, []):
        result = dfs_limited(child, goal, depth_limit - 1, graph)
        if result is not None:
            return [node] + result

    return None


def iterative_deepening_search(start, goal, graph):
    depth = 0
    while True:
        print(f"Trying depth limit: {depth}")
        result = dfs_limited(start, goal, depth, graph)
        if result is not None:
            return result
        depth += 1


# Example graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
```

```
    'F': [],
    'G': []
}

# Run IDS
path = iterative_deepening_search('A', 'G', graph)
print("Goal found! Path:", path)
```

Flowchart:



Output:

```
Enter "help" below or click "Help" above for more inform
>>>
===== RESTART: C:/Users/hemal/OneDrive/Documents/SIMATS/
Trying depth limit: 0
Trying depth limit: 1
Trying depth limit: 2
Trying depth limit: 3
Trying depth limit: 4
Goal found! Path: ['A', 'B', 'E', 'G']
```

Result:

The IDS algorithm successfully finds the goal node **G** starting from **A**, exploring depth levels one by one. The final path returned is:

A → B → E → G

This confirms that IDS correctly performs repeated depth-limited searches and finds the optimal depth at which the goal exists.